Exam — Object-Oriented Programming

1. Theoretical Part: 60min

Task A — Basic Concepts 10 P.

Explain the following five concepts from object-oriented programming and provide an example for each:

- *Encapsulation*
- *Polymorphism*
- *Constructor*
- *Interface*
- *Abstract Class*

1. *Encapsulation is the principle of bundling data and methods that operate on that data within a class, while hiding internal implementation details through access modifiers.*

```java
private double balance;

public double getBalance() {
  return balance;
}

public void setBalance(double balance) {
  this.balance = balance;
}
```

2. *Polymorphism allows objects of different classes to be treated through a common interface, enabling methods to behave differently based on the object type.*

```java
class Animal {
  public void makeSound() {
    System.out.println("...");
  }
};
class Cat extends Animal {
  public void sleep() {
    System.out.println("Purr");

  }
}
```

3. *Constructor is a special method called when creating an object instance, used to initialize the object's state and allocate resources.*

```java
};public abstract void makeSound();
abstract class Animal {
```

4. *Abstract class is a class that cannot be instantiated and may contain abstract methods (without implementation) and concrete methods (with implementation).*
5. *Interface is a contract that defines a set of abstract methods that implementing classes must provide, enabling polymorphism and multiple inheritance of type.*

Task B — Concepts of Object-Oriented Programming 15 P.

Briefly describe what the various terms and concepts mean in Java or object-oriented programming.

1. Explain how inheritance works in Java and what role the `super` keyword plays. Provide an example showing the use of `super` in both constructors and methods. 5 P.

Inheritance allows a subclass to inherit attributes and methods from a parent class using the `extends` keyword. The subclass can reuse and extend the functionality of the parent class.

The **`super` keyword** refers to the parent class and is used to:

- Call the parent class constructor: `super()`
- Access parent class methods: `super.methodName()`

```java
class Animal {
    protected String name;

    public Animal(String name) {
        this.name = name;
    }

    public void makeSound() {
        System.out.println("Some sound");
    }
}

class Dog extends Animal {
    private String breed;

    public Dog(String name, String breed) {
        super(name);  // Call parent constructor
        this.breed = breed;
    }

    @Override
    public void makeSound() {
        super.makeSound();  // Call parent method
        System.out.println("Woof!");
    }
}
```

2. Explain the difference between method overloading and method overriding. When is each used, and what are the rules for each? Provide examples. 5 P.

Method Overloading (compile-time polymorphism):

- Same class, same method name, **different parameters**
- Return type can be different

```java
public int add(int a, int b) { return a + b; }
public double add(double a, double b) { return a + b; }
```

Method Overriding (runtime polymorphism):

- Subclass provides different implementation of parent method
- **Same signature** (name, parameters, return type)
- Use `@Override` annotation

```java
class Animal { public void speak() { System.out.println("..."); } }
class Cat extends Animal {
    @Override
    public void speak() { System.out.println("Meow"); }
}
```

3. Describe the three main access modifiers in Java: public, private, and protected. How do they control visibility, and when should each be used? Provide examples. 5 P.

`public`: Accessible from **anywhere** (any class, any package).

- Use for: APIs, methods meant to be called externally

```java
public void displayInfo() { ... }
```

`private`: Accessible only **within the same class**.

- Use for: Internal data, encapsulation of attributes

```java
private double balance;  // Only accessible via getters/setters
```

`protected`: Accessible within the **same class, same package, and subclasses** (even in different packages).

- Use for: Members that subclasses need to access

```java
protected String name;  // Subclasses can access directly
```

Task C — True or False 16 P.

Decide whether the following statements are true or false:

Question	***True***	***False***
In Java, a subclass constructor must call a superclass constructor either explicitly or implicitly.	*x*	
Two methods with the same name and parameters but different return types can coexist in the same class.		*x*
The protected access modifier allows access from any class in the same package.	*x*	
An abstract class must contain at least one abstract method.		*x*
Interface methods are public and abstract by default.	*x*	
A concrete class that implements an interface must provide implementations for all interface methods.	*x*	
The `super` keyword can be used to access private members of the parent class.		*x*
A static method can access instance variables directly without an object reference.		*x*

Task D — Code Completion 12 P.

Complete the following `Circle` class by filling in the missing code directly in the gaps. The class should represent a circle with a radius. Use `Math.PI` for π.

```java
public class Circle {
    // TODO: Add a private attribute for radius (double)
    private double radius;

    // TODO: Create a constructor that takes radius as a parameter
    // and initializes the attribute
    public Circle(double radius) {
      this.radius = radius;
    }

    // TODO: Create a getter method for radius
    public double getRadius() {
      return radius;
    }

    // TODO: Create a setter method for radius that validates the radius is positive
    public void setRadius(double radius) {
      if (radius > 0.0) {
        this.radius = radius;
      }
    }

    // TODO: Create a method calculateArea() that returns the area (π × r²)
    public double calculateArea() {
      return Math.PI * radius * radius;
    }

    // TODO: Create a method calculateCircumference() that returns the circumference (2 × π × r)
    public double calculateCircumference() {
      return 2 * Math.PI * radius;
    }

    // TODO: Override the toString() method to return a string in the format:
    // "Circle[radius=5.0]"
    public String toString() {
      return "Circle[radius=" + radius + "]"
    }
```

```
38 }
```

Task E — Code Debugging 12 P.

The following `Student` class contains several errors. Find and explain all the errors in the code below. Note that errors include both syntax errors that would prevent compilation AND violations of OOP principles (such as encapsulation).

```java
public class Student {
    String name;
    private int studentId;
    private double gpa;

    public Student(String name, int studentId) {
        this.name = name;
        this.studentId = studentId
    }

    public void setGpa(double gpa) {
        gpa = gpa;
    }

    public int getStudentId() {
        return this.studentId;
    }

    public static String getStudentInfo() {
        return "Student: " + name + ", ID: " + studentId + ", GPA: " + gpa;
    }

    public void displayInfo() {
        System.out.println(getStudentInfo());
    }
}
```

List all errors you found and explain what is wrong:

- Error 1: name attribute has no access modifier (package-private) - violates encapsulation, should be private
- Error 2: Constructor missing semicolon after `this.studentId = studentId`
- Error 3: Constructor doesn't initialize `gpa` attribute - all attributes should be initialized
- Error 4: In `setGpa`, `gpa = gpa` assigns parameter to itself - should use `this.gpa = gpa`
- Error 5: `getStudentInfo()` is static but tries to access instance variables (`name`, `studentId`, `gpa`) - static methods cannot access instance variables directly
- Error 6: Missing getter methods for `name` and `gpa` - breaks encapsulation principle of providing access to private data

2. Practical Part: 60min

Task F — Contact Book Application 40 P.

Create a contact book application that allows users to manage a list of contacts. This task requires implementing multiple classes and demonstrating object-oriented programming principles.

Your program should support the following operations:

- *Add a new contact with name, phone number, and email*
- *Delete a contact by name*
- *Search for a contact by name (case-insensitive)*
- *Display all contacts*

The application should use proper OOP design with separate `Contact` and `ContactBook` classes.

> 🔥 ***Tip***
>
> *You can use `ArrayList<Contact>` to store contacts. Example:*
>
> ```java
> import java.util.ArrayList;
>
> ArrayList<Contact> contacts = new ArrayList<>();
> contacts.add(new Contact("John", "123-456", "john@email.com"));
> ```
>
> *For searching, use `String.equalsIgnoreCase(String other)` for case-insensitive comparison.*

1. Create a new project. Give the project a name that contains your student ID number and your name. Create both a `Contact` class and a `ContactBook` class. 3 P.

2. Implement the `Contact` class with private attributes for name, phone, and email. Include a constructor that initializes all three attributes and getter methods for each attribute. Override the `toString()` method to provide a readable representation. 5 P.

3. Implement the `ContactBook` class with a private `ArrayList<Contact>` to store contacts. Include a constructor that initializes the empty list. 4 P.

4. Create an `addContact(String name, String phone, String email)` method in ContactBook that creates a new Contact object and adds it to the list. Validate that none of the parameters are null or empty before adding. 6 P.

5. Create a `deleteContact(String name)` method that removes a contact by name (case-insensitive). Return true if a contact was deleted, false if no matching contact was found. 5 P.

6. Create a `searchContact(String name)` method that searches for and returns a contact by name (case-insensitive). Return null if no matching contact is found. 6 P.

7. Create a `displayAllContacts()` method that prints all contacts in a readable format. Handle the case when the contact book is empty. 3 P.

8. Create a `main` method that demonstrates all functionality comprehensively. Add several contacts, test searching, deleting, and displaying. Show clear output for each operation. 5 P.

9. When programming your solution, pay attention to proper coding style including naming conventions, encapsulation (private attributes with public methods), and clear code structure. 3 P.

In total, 105 + 0 pts. are achievable. You have achieved _____ pts. out of 105 pts.

Points	105–94	93–84	83–73	72–63	62–0
Value	very good	good	satisfactory	sufficient	failed

Solution Proposals – Exam

Task	Points Achieved
Task A — Basic Concepts	____ / **10**
Encapsulation is the principle of bundling data and methods that operate on that data within a class, while hiding internal implementation details through access modifiers.	____ / 2
Polymorphism allows objects of different classes to be treated through a common interface, enabling methods to behave differently based on the object type.	____ / 2
Constructor is a special method called when creating an object instance, used to initialize the object's state and allocate resources.	____ / 2
Interface is a contract that defines a set of abstract methods that implementing classes must provide, enabling abstraction and multiple inheritance of type.	____ / 2
Abstract Class is a class that cannot be instantiated and may contain both abstract methods (without implementation) and concrete methods (with implementation).	____ / 2
Task B — Concepts of Object-Oriented Programming	____ / **15**
Inheritance allows a subclass to inherit attributes and methods from a parent class. The `super` keyword refers to the parent class and is used to call parent constructors (super()) or access parent methods (super.method()). This enables code reuse and specialization.	____ / 5
Method overloading occurs in the same class with methods having the same name but different parameters (compile-time polymorphism). Method overriding occurs in subclasses providing a different implementation of a parent method with the same signature (runtime polymorphism).	____ / 5
Public members are accessible everywhere. Private members are only accessible within the same class. Protected members are accessible within the same class, subclasses, and the same package. Use private for encapsulation, protected for inheritance, and public for interfaces.	____ / 5
Task C — True or False	____ / **16**
True	____ / 2
False	____ / 2
True	____ / 2
True	____ / 2
False	____ / 2
False	____ / 2
Task D — Code Completion	____ / **12**
Private double attribute: radius	____ / 2
Constructor with one parameter that initializes radius using `this` keyword	____ / 2
Getter method: getRadius() that returns the radius value	____ / 2
Setter method: setRadius() that validates radius > 0 before setting	____ / 2
calculateArea() method returns Math.PI **radius** radius	____ / 2
calculateCircumference() method returns 2 **Math.PI** radius	____ / 2
Task E — Code Debugging	____ / **12**
Error 1: name attribute has no access modifier (package-private) - violates encapsulation, should be private	____ / 2
Error 2: Constructor missing semicolon after `this.studentId = studentId`	____ / 2
Error 3: Constructor doesn't initialize gpa attribute - all attributes should be initialized	____ / 2
Error 4: In setGpa, `gpa = gpa` assigns parameter to itself - should use `this.gpa = gpa`	____ / 2
Error 5: getStudentInfo() is static but tries to access instance variables (name, studentId, gpa) - static methods cannot access instance variables directly	____ / 2
Error 6: Missing getter methods for name and gpa - breaks encapsulation principle of providing access to private data	____ / 2
Task F — Contact Book Application	____ / **40**

Project created with appropriate name. Both Contact and ContactBook classes exist and are properly structured.	____ / 3
Contact class properly implemented with private attributes, constructor, getters, and toString() override.	____ / 5
ContactBook class properly implemented with ArrayList and constructor.	____ / 4
addContact method works correctly with proper validation of inputs.	____ / 6
deleteContact method works correctly with case-insensitive search and returns appropriate boolean.	____ / 5
searchContact method works correctly with case-insensitive matching and returns correct Contact or null.	____ / 6
displayAllContacts method works correctly and handles empty contact book gracefully.	____ / 3
Main method demonstrates all functionality comprehensively with multiple test cases.	____ / 5
Code follows proper OOP principles: encapsulation, good naming, clear structure, and appropriate comments.	____ / 3
	____ / 105 + 0 P.